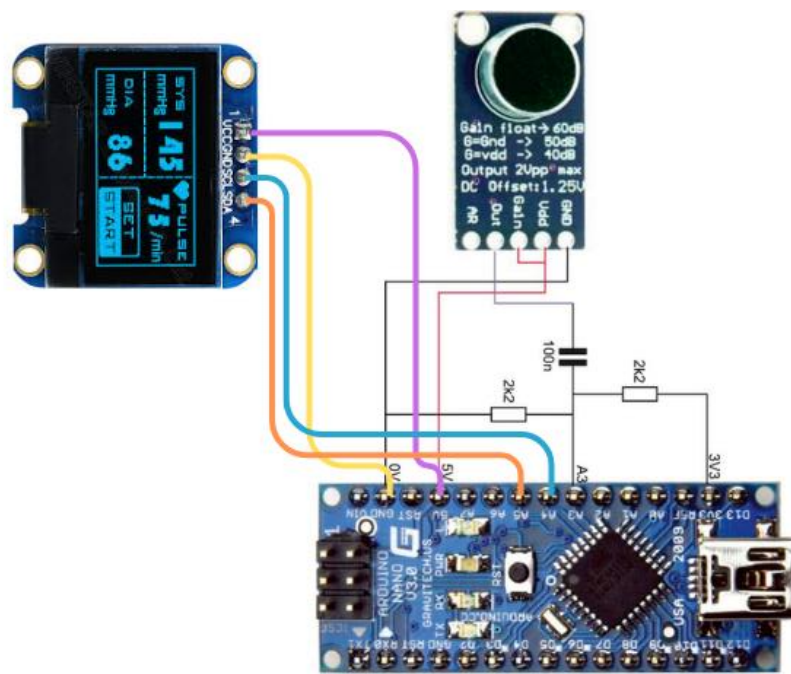
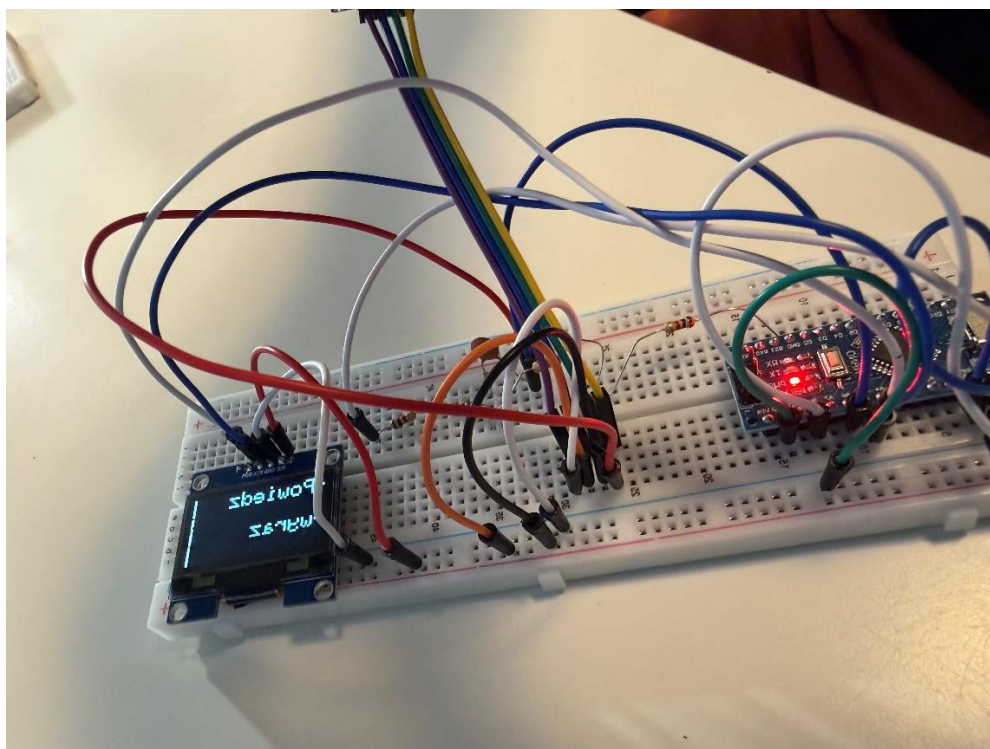


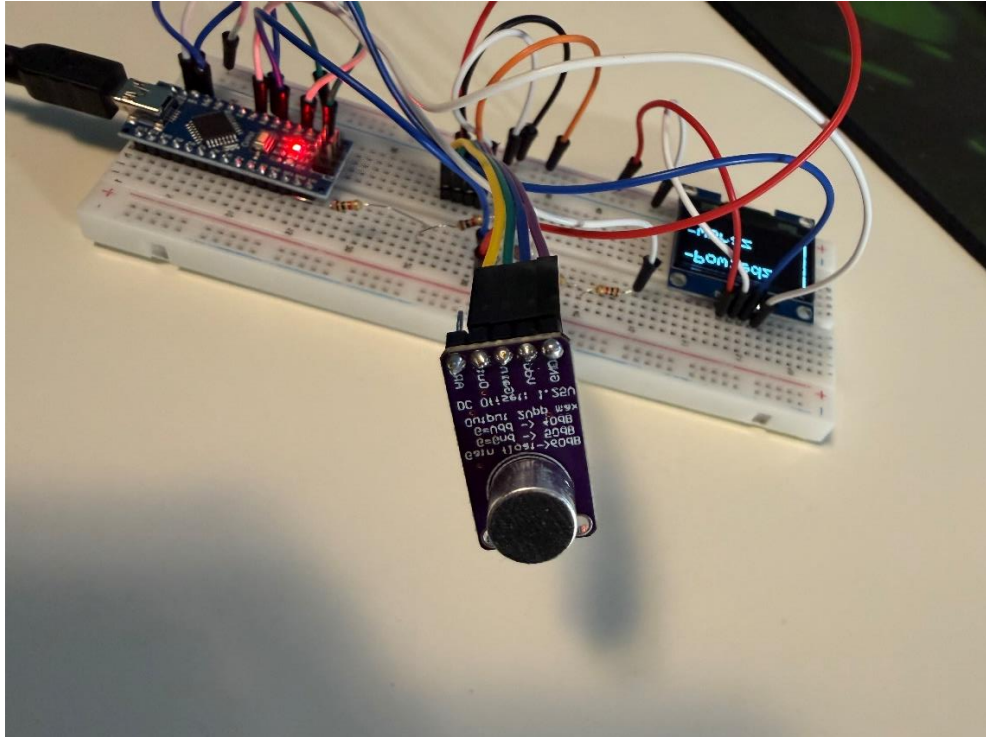
Łukasz Śmichurski

1. Schemat układu



2. Zdjęcia układu





3. Koncepcja działania

Układ działa w pełni autonomicznie, bez konieczności podłączenia do komputera. Dźwięk zbierany z mikrofonu jest próbkowany przez przetwornik ADC wbudowany w Arduino. Algorytm w czasie rzeczywistym analizuje sygnał pod kątem jego amplitudy. Gdy głośność przekroczy ustalony próg, układ zaczyna rejestrować słowo, dzieląc je na 13 równych segmentów czasowych.

Dla każdego segmentu wyodrębniane są cechy charakterystyczne dźwięku za pomocą cyfrowych filtrów pasmowych oraz analizy przejść przez zero. Wynikiem analizy jest odcisk nagrałego słowa w postaci macierzy liczb.

Następnie układ porównuje ten odcisk z 10 zapisanymi w pamięci programu wzorcami za pomocą algorytmu dopasowywania wzorców (uwzględniającego przesunięcia czasowe i odchylenie standardowe). Wzorzec o najmniejszym dystansie od nagranej próbki jest uznawany za rozpoznane słowo. Ostatnim krokiem jest wyświetlenie rozpoznanego słowa na ekranie OLED.

4. Program

Oprogramowanie bazuje na rozwiązaniu autorstwa Petera Balcha (z serwisu Instructables), które zostało zmodyfikowane. Dodano sprzężenie zwrotne za pomocą wyświetlacza OLED (informowanie użytkownika o stanach: "Powiedz wyraz", "Słucham...", oraz "-Wynik:"). Układ został całkowicie uniezależniony od portu szeregowego w normalnej pracy operacyjnej.

```

1 //-----
2 // Copyright 2021 Peter Balch
3 // recognise words
4 // subject to the GNU General Public License
5 //-----
6
7 #include <Arduino.h>
8 #include <SPI.h>
9 #include "Coeffs.h"
10 #include "Templates.h"
11
12 #include <Wire.h>
13 #include "SSD1306Ascii.h"
14 #include "SSD1306AsciiWire.h"
15
16 #define I2C_ADDRESS 0x3C
17 SSD1306AsciiWire oled;
18
19 unsigned long wordShowTime = 0;
20 byte currentState = 0;
21
22
23 void showOLED(const char* line1, const char* line2 = "") {
24     oled.clear();
25     oled.set2X();
26     oled.println(line1);
27     oled.println(" ");
28     if (line2[0] != '\0') {
29         oled.println(line2);
30     }
31 }
32
33 //-----
34 // GetSerial
35 //-----
36 byte GetSerial() {
37     while (Serial.available() == 0);
38     return Serial.read();
39 }
40
41 //-----
42 // PollBands
43 //-----
44 bool PollBands(bool init)
45 {
46     bool IsPos, Collecting;
47     static unsigned long prevTime;
48     byte band, seg, val1, val2, i;
49     const byte hyster = 20;
50     static int zero = 500;
51     static byte curSegment = 255;
52     long val;
53     word zcr;
54     static int valmax[10];
55     static int pd[10] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
56     static int ppd[10] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
57     static int ppval = 0;
58     static int pval = 0;
59
60     if (val < zero)
61         zero--;
62     else
63         val = val - zero;
64
65     if (!Collecting) {
66         if (abs(val) > thresh)
67             Collecting = true;
68         else
69             return false;
70     }
71
72     if (IsPos)
73     {
74         if (val < -hyster)
75         {
76             IsPos = false;
77             zcr++;
78         }
79         else {
80             if (val > +hyster)
81             {
82                 IsPos = true;
83                 zcr++;
84             }
85         }
86         ppval = pval;
87         pval = val;
88
89         for (band = 0; band < nband; band++)
90         {
91             int i1, i2;
92             i1 = ((-filt_bi[band]) * pd[band] - filt_bo[band] * ppd[band]) >> 16;
93             i2 = (filt_ai[band] * i1 - filt_aj[band] * ppd[band]) >> 16;
94             ppd[band] = pd[band];
95
96             i = FindBestUtterance(Utterance, &i1);
97             strcpy_P(buffer, (char *)pgm_read_word(&(utterances[i])));
98             Serial.println(i);
99             Serial.println(buffer);
100
101             // Budujemy nowy ciąg znaków: myślnik + oryginalny wyraz
102             sprintf(displayBuffer, "-%s", buffer);
103
104             // --- Po zakończeniu liczenia pokazujemy wyraz na ekranie ---
105             showOLED("Wytnik:", displayBuffer);
106             wordShowTime = millis();
107             currentState = 1; // Zakonczony stan
108         }
109     }
110
111     //-----
112     // PrintCurBandData
113     //-----
114     void PrintCurBandData(void)
115     {
116         byte seg, band;
117
118         Serial.println("a");
119         for (seg = 0; seg < nSegments; seg++) {
120             for (band = 0; band <= nBand; band++) {
121                 Serial.print(curBandData[seg][band]);
122                 Serial.print(" ");
123             }
124             Serial.println("");
125         }
126     }
127
128     //-----
129     // SendUtterance
130     //-----
131     if (init)
132     {
133         memset(pd, 0, sizeof(pd));
134         memset(ppd, 0, sizeof(ppd));
135         memset(curBandData, 0, sizeof(curBandData));
136         pval = 0;
137         ppval = 0;
138         return false;
139     }
140
141     val = 0;
142     IsPos = true;
143     Collecting = false;
144     int n = 0;
145
146     for (curSegment = 0; curSegment < nSegments; curSegment++) {
147         zcr = 0;
148         prevTime = 0;
149         memset(valmax, 0, sizeof(valmax));
150
151         prevTime = millis();
152         while (millis() - prevTime < SegmentSize)
153         {
154             while (!getBit(ADCSRA, ADIF)) ; // wait for ADC
155             val1 = ADC1;
156             val2 = ADC0;
157             bitSet(ADCSRA, ADIF); // clear the flag
158             bitSet(ADCSRA, ADSC); // start ADC conversion
159             val = val1;
160             val += val2 << 8;
161
162             pd[band] = pd[band];
163             p[band] = i1;
164             if (abs(i2) > valmax[band])
165                 valmax[band]++;
166         }
167
168         for (band = 0; band < nBand; band++)
169             curBandData[curSegment][band + 1] = valmax[band];
170         curBandData[curSegment][0] = zcr;
171     }
172
173     if (collecting) {
174         Serial.println("a");
175         SendUtterance(curBandData);
176
177         // --- Zakonczony ekran na "Slucham..." podczas liczenia ---
178         if (!bAnalyze) {
179             showOLED("Slucham...");
180             AnalyzeUtterance(curBandData);
181         }
182     }
183
184     return Collecting;
185 }
186
187 //-----
188 // AnalyseUtterance
189 //-----
190 void AnalyseUtterance(int Utterance[nSegments][nBand+1]) {
191     int i, dist;
192     char buffer[10];
193     char displayBuffer[30]; // Dodatkowy bufor na wyraz z myślnikiem
194     i = FindBestUtterance(Utterance, &dist);
195
196     //-----
197     // ShiftUtterance
198     //-----
199     int ShiftUtterance(int Utterance[nSegments][nBand+1], byte TemplateUttr, int8_t shift) {
200         byte band, seg, importance;
201         int aUtterance[nSegments][nBand+1];
202         int Dist, aMean, aSD;
203         ShiftUtterance(Utterance, aUtterance, shift);
204
205         Dist = 0;
206         for (seg = 0; seg < nSegments; seg++) {
207             if (seg == 0)
208                 importance = 2;
209             else
210                 importance = 1;
211
212             for (band = 0; band <= nBand; band++) {
213                 aMean = pgm_read_word(&Templates[TemplateUttr][seg][band].mean);
214                 aSD = pgm_read_word(&Templates[TemplateUttr][seg][band].sd);
215                 Dist = constrain(Dist + importance * abs(((long)aUtterance[seg][band]) - aMean) * 1000 / (50 * aSD), -10000, +10000);
216             }
217         }
218         return Dist;
219     }
220
221     //-----
222     // ShiftUtterance
223     //-----
224     void ShiftUtterance(int utSource[nSegments][nBand+1], int utDest[nSegments][nBand+1], int shift) {
225         int8_t i, j, k, n;
226         int m;
227     }

```

```

260 void ShiftUtterance(int utSource[nSegments][nBand+1], int utDest[nSegments][nBand+1], int shift) {
261     int i,j,k,n;
262     int m;
263     byte seg,band;
264
265     if (shift == 0) {
266         for (seg = 0; seg < nSegments; seg++)
267             for (band = 0; band <= nBand; band++)
268                 utDest[seg][band] = utSource[seg][band];
269     } else {
270         for (seg = 0; seg < nSegments; seg++)
271         {
272             n = SubShifts*(nSegments-1);
273             i = shift*(nSegments-1);
274             j = -i / (SubShifts*(nSegments-1));
275             i = abs(i) % ((nSegments-1)*SubShifts);
276
277             if (shift < 0)
278                 k = j+1; else
279                 k = j-1;
280             for (band = 0; band <= nBand; band++) {
281                 if ((seg+j >= 0) && (seg+j < nSegments))
282                     n = utSource[seg+j][band]*((nSegments-1)*SubShifts-1) / n; else
283                     n = 0;
284                 if ((seg+k >= 0) && (seg+k < nSegments))
285                     n = m+utSource[seg+k][band]*i / n;
286                 utDest[seg][band] = n;
287             }
288         }
289     }
290     NormaliseUtterance(utDest);
291 }
292
293
390 int FindBestUtterance(int Utterance[nSegments][nBand+1], int *BestDist) {
391     int dist,BestUtDist,shiftDist;
392     int BestUtt,shift,TemplateUtt;
393
394     BestUtt = 0;
395     *BestDist = 0;
396     BestUtDist = 0xFFFF;
397     for (TemplateUtt = 0; TemplateUtt < nUtterances; TemplateUtt++) {
398         shift = FindBestShift(Utterance,TemplateUtt);
399         shiftDist = ShiftedDistance(Utterance,TemplateUtt,shift);
400
401         if (shiftDist < BestUtDist) {
402             BestUtDist = shiftDist;
403             BestUtt = TemplateUtt;
404             *BestDist = BestUtDist;
405         }
406     }
407     return BestUtt;
408 }
409
410 //-----
411 // CheckSerial
412 //-----
413 void CheckSerial(void)
414 {
415     byte seg,band,i,j;
416     static int Utterance[nSegments][nBand+1];
417
418     if (Serial.available() > 0) {
419         switch (GetSerial()) {
420             case 'u':
421                 for (seg = 0; seg < nSegments; seg++) {
422                     for (band = 0; band <= nBand; band++) {
423                         Utterance[seg][band] = GetSerial();
424                         Utterance[seg][band] += 256*GetSerial();
425                     }
426                 }
427             }
428         }
429     }
430 }
431
432 //-----
433 // loop
434 //-----
435 void loop(void)
436 {
437     CheckSerial();
438     PollBands(false);
439
440     // Sprawdzamy, czy minęły 2 sekundy od pokazania wyniku
441     if (currentState == 1 && (millis() - wordShoutLine >= 12000)) {
442         currentState = 0;
443         showOLED("Powiedz", "wyraz");
444     }
445 }

```

5. Opis treningu

Trening układu polegał na przygotowaniu 10 słów: URUCHOM, DOM, POCZĄTEK, DZIĘSIEĆ, JEDEN, LUSTRO, SZAFA, PRAWO, MYSZ, SZNUR.

Proces treningu składał się z następujących kroków:

- Zbiórka danych: Za pomocą specjalnego trybu w programie bazowym podłączonym do komputera, wielokrotnie wypowiadano do mikrofonu wybrane słowa. Arduino przesyłało wyodrębnione macierze cech (segmenty x pasma) przez port szeregowy na komputer.
- Generowanie wzorców: Otrzymane dane liczbowe dla każdego z 10 słów zostały przetworzone (uśrednione z uwzględnieniem odchylenia standardowego).
- Implementacja w kodzie: Wygenerowane średnie (mean) i odchylenia (sd) zostały zamienione w stałe tablice w pliku nagłówkowym Templates.h (przechowywane w PROGMEM, aby oszczędzać pamięć operacyjną). Słownik pojęć został zaktualizowany na wybrane polskie odpowiedniki. Dzięki temu, podczas działania, mikrokontroler po prostu porównuje w czasie rzeczywistym to, co słyszy, z tablicą wgranych na stałe wartości statystycznych.